
Machine Learning for Healthcare - Project Report

Samuel Gair
22-927-362

Nicolas Balke
21-560-107

Finn Brunke
21-950-274

1 Data Processing and Exploration

1.1 Data Transformation

We merged all patient records into a single Parquet file to strictly preserve temporal causality and prevent data leakage, timestamps were rounded to the next full hour so future information was never available at an earlier time.

1.2 Exploratory Data Analysis

Parameter distributions vary significantly: variables like BUN, creatinine, and PaO2 are strongly right-skewed, while Albumin follows a rough bell curve. Demographically, most patients in the dataset are over 45. Categorical parameters are numerically encoded (e.g., Gender and In-hospital death as 0/1, ICU Type as 1-4). Correlations emerged during demographic filtering: for ICU types 2-4, the most common GCS score is 14, but this shifts to 16 when ICU type 1 is included. Additionally, average urine output drops for the 40-80 age bracket compared to the overall population average.

1.3 Preprocess Data for Machine Learning

We initially applied a forward-filling imputation scheme, assuming unmeasured values remain stable. However, this is clinically flawed—if a patient’s heart rate spikes but isn’t measured, the model falsely assumes it stayed constant. For parameters completely lacking a history, we imputed with -1 (since clinical values are non-negative) or used a mean imputer. Because our EDA showed many parameters follow a Gaussian distribution, using scikit-learn’s SimpleImputer and RobustScaler (which uses medians to resist outliers) effectively preserved natural data shapes.

2 Supervised Learning

2.1 Classic Machine Learning Methods

We established our baselines using compressed static features to predict outcomes.

Model	Feature Engineering Strategy	AUROC	AUPRC
Logistic Regression	Simple Means	0.8223	0.4419
Random Forest	Simple Means	0.8094	0.4352
Random Forest	Min / Max / Mean aggregations	0.8237	0.4370
Random Forest	tsfresh temporal extraction	0.8219	0.4641

Table 1: Performance of Classic Machine Learning Methods

Classifier Evaluation (Pros & Cons): Logistic Regression is fast and interpretable, but struggles with complex, non-linear relationships. Random Forest naturally captures non-linearities and handles unscaled data well, but acts as a "black box" and risks overfitting on high-dimensional data. Adding temporal features (tsfresh or min/max bounds) provided these static models with crucial context about patient trends that simple averaging destroys, yielding a noticeable boost in AUPRC.

2.2 Recurrent Neural Networks

For our neural network architectures, we transitioned to sequential processing.

Architecture	Data Representation	AUROC	AP
LSTM	Imputed Time-Grid	0.5000	0.1462
BiLSTM	Imputed Time-Grid	0.5053	0.1484
Transformer	Imputed Time-Grid	0.5199	0.1538
Transformer	Tokenized Triplets (t, z, v)	0.7370	0.2998

Table 2: Performance of Deep Learning Sequence Models

RNN vs. BiLSTM Architecture: For the LSTM, we used the final timestep’s output to leverage the entire sequence history. However, standard RNNs often "forget" early signals. Furthermore, standard time-series forecasting prohibits looking ahead to avoid "cheating." But since we are predicting an event occurring *after* a fixed 48-hour window, processing the data bidirectionally (BiLSTM) is valid and provides the network with richer temporal context.

Transformers vs. RNNs: Unlike RNNs, Transformers process the entire sequence simultaneously using self-attention. This parallel architecture completely bypasses the "forgetting" issue of RNNs and significantly accelerates training.

2.3 Tokenizing Time-Series Data and Transformers

The Triplet Transformer vastly outperformed the imputed baselines. Imputing the time-grid essentially injected noise and forward-filling falsely assumes a patient’s vitals are perfectly stable just because they weren’t measured. The (t, z, v) triplet approach fixes this by relying strictly on actual ground-truth measurements. It also preserves exact timestamps, letting the model explicitly exploit "informative missingness" (recognizing that a doctor ordering a test at a specific time is a strong clinical signal in itself).

3 Representation Learning

3.1 Pretraining and Linear Probes

We pretrained multiple different encoders based on tokenized measurements as in Q2.3b. We developed an architecture that combined the InfoNCE approach, a TS2Vec-style contrastive term, hard negative mining and a time-frequency consistency term. In addition we trained a non-contrastive encoder-decoder version, using only cross-entropy loss on reconstructions, and a hybrid version, in which both paths are included. We then monitor this pretraining step by jointly looking at the contrastive and reconstruction losses, as well as computing quantitative clustering metrics and some collapse detection metric every few epochs. After pretraining we fix the learned encoder and train a Logistic Regression on the embeddings, which lead to the following results:

Table 3: Representation Learning Results (full-dataset)

	Contrastive	Hybrid	Auto-encoder
AuROC	0.814	0.806	0.822
AuPRC	0.374	0.351	0.461

This suggests that the contrastive approaches are too complex in this case and a simpler, auto-encoder based, approach provides the most generalization to the test data. We hypothesize that in higher data regimes, or with more ablation studies on the exact augmentations and contrastive approaches, these could exceed the auto-encoder approach.

3.2 Simulating Label Scarcity

To compare the representation learning approach with classical machine learning, we train the classical models as well as the linear probe on 100, 500 and 1000 patients respectively and report the full test-set performance.

Table 4: Representation Learning Results AuROC and AuPRC (label-scarcity)

Patients	Repr. learning (best)		Random Forest		LSTM		Transformer	
	AuROC	AuPRC	AuROC	AuPRC	AuROC	AuPRC	AuROC	AuPRC
100	0.706	0.225	0.754	0.364	0.685	0.271	0.701	0.300
500	0.736	0.353	0.816	0.394	0.724	0.326	0.787	0.378
1000	0.779	0.355	0.836	0.501	0.764	0.363	0.777	0.313

This suggests that simple models are better suited for this task and representation learning can only start to approach the performance, once the linear probe has enough data points.

3.3 Visualizing Learned Representations

We visualize the learned representations in t-SNE and UMAP dimensionality-reduced spaces (see Figure 1 in the Appendix).

Upon visual inspection, data-points with different labels do not seem to be distributed identically, even though there is no clear separation. We make this notion more precise by employing some quantitative clustering metrics. We decided to use the silhouette score, centroid distance and Davies-Bouldin index, reported below.

Table 5: Quantitative Clustering Metrics

	Contrastive	Hybrid	Auto-encoder
Silhouette	0.006	0.016	0.005
Centroid Distance	0.237	2.051	3.068
Davies-Bouldin	6.706	4.703	5.601

These results hint that, even though the auto-encoder generalizes best, in terms of separation of clusters, which can aid the interpretability of the model, a hybrid approach seems to outperform the auto-encoder.

4 Foundation Models

We prompted both gemma3:1b and llama3.1:latest to predict hospital deaths of patients. Later we used the embeddings of Ollama models to train a small linear probe and compared the results to using pretrained timeseries chronos-forecasting models’ embeddings.

4.1 Prompting an LLM to Solve a Time-Series Problem

To keep the prompt short we extracted critical parameters like maximum temperature, maximum heartrate or urine amount. A few shot approach was used where 7 datapoints from the train set were included alongside each patient. Each prompt tasked llama with predicting the survival of the hospital stay and to give a probability of death 0-1.

As can be seen, in Table 4 the small gemma3.1:b model predicts arbitrarily anything with an AUROC of 0.502. The larger llama3.1:latest is slightly better with 0.597 but still close to random.

4.2 Using LLMs to Retrieve Embeddings

Using llama3.1:latest we retrieved embeddings for the prompts and used these embeddings to train a small linear probe.

Table 6: Using LLama models to predict "in-hospital_death"

Name	AUROC	AUPRC	Log Loss	Brier Score
gemma3.1:b	0.502	0.143	1.276	0.322
llama3.1:latest	0.597	0.189	0.760	0.273

From our experiments supervised models performed better, scoring 0.8 AUROC, than linear probes on llama’s embeddings. The representation model from section 3 clearly outperforms the linear probe on the embeddings of llama.

As can be seen in Figure 2 (in the Appendix) LLamas embeddings are mostly just a point cloud, although t-SNE and UMAP show some structure. Chronos seems to create 2 very distinct clusters.

4.3 Using Time-Series Foundation Models

Using a channel Transformer we transformed the timeseries data into a 1D-vector. This new dataset was used on the base model of the chronos family, to get embeddings from a pretrained timeseries model.

For the embedding both pooling the channels-mean and just selecting the last layer was tried, with the latter giving slightly better results.

Table 7: Linear Probe trained on embeddings

Name	Description	AUROC	AUPRC	Log Loss	Brier Score
llama embeddings	C = 0.001	0.688	0.279	0.423	0.131
chronos embeddings	C = 0.001	0.741	0.352	0.459	0.131

Finally we trained a small neural network on the chronos embeddings. For this we use a self-attention layer into a softmax followed by a classifier. This way the model learns to weight some channels more than others.

Table 8: Small self attention probe trained on chronos.base embeddings

Name	AUROC	AUPRC	Log Loss	Brier Score
Self-Attention probe	0.767	0.382	0.433	0.125

5 General Questions

5.1 Yes, our classic models like Random Forest (AUROC 0.82) consistently outperformed the deep learning architectures in this project. Deep models tend to overfit on sparse, noisy clinical data, whereas classic methods efficiently extract robust signals without needing massive datasets.

5.2 Self-attention scales quadratically challenging transformer models. So every step dilutes the important points, because all steps get the same amount of attention. LSTM’s should be well suited to handle these long time series since they have a strong temporal inductive bias and can remember sequential dependencies. Probably best suited are autoencoders, they can reduce the information down to the most important parts, in the process reduce noise and are able to capture global structures in the data.

5.3 Self-supervised learning is well-suited to medical settings, where patient data is abundant but outcome labels are difficult to obtain. However, applying it to time-series data remains challenging. Data augmentations used to form positive and negative pairs can have inconsistent medical interpretations across samples, as features differ in semantics and measurement frequency. Additionally, integrating static and dynamic data requires careful handling. Methods effective for fine-grained time series, such as time-frequency consistency, are often unsuitable in medical contexts due to sparse and irregular sampling, which limits the usefulness of frequency-based representations.

121 **6 Appendix**

122 **6.1 Visualizations**

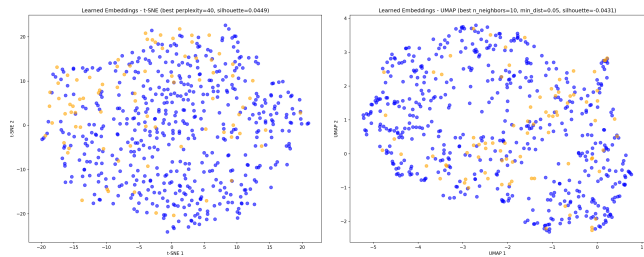


Figure 1: T-SNE and UMAP visualization of learned embeddings (Auto-encoder).

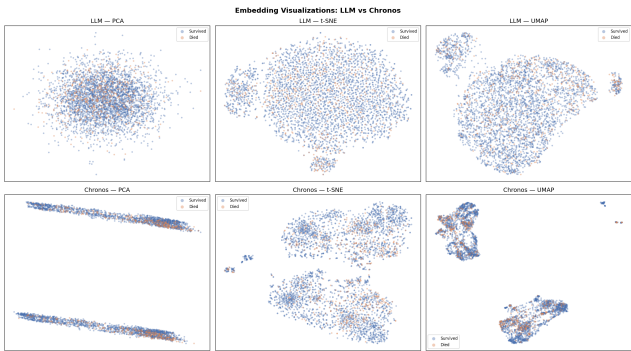


Figure 2: T-SNE, PCNE and UMAP visualizations of the embeddings from llama3.1:latest and chronos.base

123 **6.2 Tables**

Table 9: Linear Probe trained on chronos.base embeddings

Name	AUROC	AUPRC	Log Loss	Brier Score
C = 0.001	0.741	0.352	0.459	0.131
C = 0.01	0.722	0.333	0.669	0.152
C = 0.1	0.7010	0.322	0.948	0.167
C = 1.0	0.700	0.315	1.139	0.172

Table 10: Linear Probe trained on lama.3.1latest embeddings

Name	AUROC	AUPRC	Log Loss	Brier Score
C = 0.001	0.688	0.279	0.423	0.131
C = 0.01	0.632	0.236	0.512	0.150
C = 0.1	0.603	0.220	0.785	0.179
C = 1.0	0.595	0.215	1.223	0.199